Semantic Layer Workflows for Adaptive Generative AI Editing in the Compute Continuum

Stelios Sotiriadis[a,*]

[a]School of Computing and Mathematical Sciences, Birkbeck, University of London, London, United Kingdom

Abstract

Interactive diffusion editing is usually an iterative workflow: users generate an image, select a region, revise it, compare variants, and roll back unwanted edits. Current pipelines often treat these steps as independent full-image generations or one-off inpainting calls, which repeats computation and can alter regions that should remain fixed.

This paper formulates semantic image editing as a workflow problem. It introduces the Semantic Layer Workflow model, where an image is represented as a semantic layer graph containing masks, bounding boxes, labels, edit history, provenance, cache keys, and execution metadata. The workflow selects among full regeneration, semantic local diffusion, cached re-editing, deterministic projection, and rollback according to edit type, latency, non-target drift, and available semantic state.

A prototype combines Stable Diffusion and SDXL checkpoints, GroundingDINO+SAM2 segmentation, local crop rendering, deterministic color projection, compositing, and persisted version state. The evaluation covers 10 diffusion checkpoints, four edit categories, and five seed variants, yielding 200 paired workflow executions at 20 denoising steps. Semantic local rendering is 1.788x faster than naive regeneration when detector overhead is excluded, but the detector-inclusive first edit averages 0.809x because semantic annotation adds 13.543 s. Cached re-edits reach 7.041x speedup, deterministic projection gives 1227x speedup for simple color edits, and rollback averages 0.188 s. A measured scheduling simulation shows semantic execution wins in 60.0% of two-edit sessions and 99.5% of five-edit sessions. The results show that semantic workflow state is most valuable for repeated, local, provenance-sensitive editing rather than universal first-edit acceleration.

Keywords: Compute continuum, Scientific workflows, Generative AI, Diffusion models, Semantic layers, Provenance, Adaptive execution, Workflow scheduling, Reproducibility, Performance modeling

1. Introduction

Generative AI pipelines are becoming a new class of data- and compute-intensive workflow. In a typical interactive image-editing session, a user does not merely request a single text-to-image generation. The user iterates: generate an image, identify a region, change a color, alter an object, undo a variant, try another version, preserve identity, and compare alternatives. Each iteration may require detector inference, segmentation, denoising, compositing, metric evaluation, and provenance updates. This process has many properties of scientific workflow execution: expensive heterogeneous tasks, data dependencies, intermediate artifacts, provenance requirements, repeated execution, and trade-offs between latency, quality, and resource cost [1–3].

Current diffusion editing systems are strong at visual synthesis but weak at workflow state. Full-image regeneration is simple, but it recomputes everything and often changes unrelated pixels. Manual inpainting gives local control, but it depends on user effort and does not by itself provide reusable semantic state. Prompt-based editing, cross-attention control, ControlNet, and instruction-tuned editors improve controllability [4–6], but most pipelines still lack a first-class representation of named image regions, edit histories, cached masks, rollback states, and policy-driven execution. The result is a mismatch between how users work and how the underlying computation is scheduled.

This paper represents iterative diffusion editing as a semantic scientific workflow. The image is treated not as a flat bitmap but as an Image DOM: a graph of named semantic layers with provenance, dependency context, and cached execution state. An edit request is then a workflow decision problem. If the requested change is a simple color update on a known layer, a deterministic projection may be sufficient. If the change requires texture and shading, semantic local diffusion may be appropriate. If the requested change is global or structurally entangled, full regeneration may be necessary. If the same layer is edited repeatedly, cached layer state can avoid repeated detector work. If a user rolls back a version, stored state can be

*Corresponding author.
Email address: s.sotiriadis@bbk.ac.uk (Stelios Sotiriadis)

restored without a new denoising run.

This framing is relevant to compute-continuum systems because the workflow exposes information that a monolithic image-generation call hides: region size, expected compute cost, cache availability, dependency risk, provenance constraints, and candidate execution operators. The present evaluation is measured on a local workstation and supplemented with a resource-sensitivity projection; it is not a deployed edge-cloud-HPC experiment. The contribution is the representation that makes such placement decisions explicit and auditable.

We make four contributions.

1. We define a semantic layer workflow model for diffusion editing, including layer graphs, edit operators, provenance records, cache keys, and version states.

2. We formulate adaptive editing as a multi-objective execution policy that trades latency, detector cost, non-target drift, expected quality loss, and user effort.

3. We implement a prototype workflow engine using Stable Diffusion and SDXL checkpoints, GroundingDINO+SAM2 masks, local crop diffusion, deterministic projection, compositing, cached re-edits, and rollback.

4. We evaluate the system with a 200-sample local benchmark across 10 models, four edit categories, and five seed variants per category. The benchmark reports latency, cached speedup, rollback time, non-target drift, inside-target change, boundary-band change, crop/mask characteristics, and a heterogeneous resource-sensitivity projection.

The central empirical result is mixed but useful. Semantic local diffusion is not always faster for a first edit once detector cost is included. Across all 200 samples, semantic rendering excluding detector overhead is 1.788x faster than naive full-image regeneration, but the detector-inclusive first edit averages only 0.809x. Once semantic state is cached, repeated layer edits achieve 7.041x average speedup. Projection-only deterministic color edits average 1227x speedup, rollback averages 0.188 s, and semantic compositing preserves non-target pixels under the implemented mask-compositing metric. The result supports semantic workflow state for locality, reproducibility, fast re-editing, and versioned execution, not a universal first-edit speedup claim.

2. Background and Motivation

2.1. Scientific workflows and provenance

Scientific workflow systems were developed to coordinate complex analyses over heterogeneous resources while preserving execution structure, intermediate artifacts, and provenance [1, 7]. Workflows make data dependencies explicit, allow repeated execution, support monitoring, and enable researchers to reason about performance and reproducibility. Their importance increases as applications span heterogeneous platforms, distributed storage, accelerators, cloud services, and edge resources [3, 8].

Generative AI editing shares these characteristics. A single session may contain multiple tasks with different resource profiles. Open-vocabulary detection and segmentation may be CPU/GPU vision tasks; diffusion denoising may require accelerator memory; deterministic compositing is lightweight; persistence and rollback are storage-bound. The workflow is also highly stateful: an edit depends on the base image, prompt, seed, model checkpoint, scheduler, semantic masks, previous variants, and user intent. Without explicit provenance, it is difficult to reproduce a result or explain why one edit changed unrelated regions.

Provenance is therefore part of the computational model. A semantic editing workflow records what was generated, which model and scheduler were used, which detector produced each mask, what parameters defined the crop, and which cached state was reused. This aligns with scientific provenance principles [9, 10] and FAIR expectations for reusable computational artifacts [11].

2.2. Diffusion models as workflow components

Denoising diffusion probabilistic models generate samples by learning a reverse denoising process from noise to data [12]. Latent diffusion models perform this process in a compressed latent space, making high-resolution text-to-image generation more efficient [13]. Let z_t be a latent at diffusion timestep t, c the conditioning, and $\epsilon_\theta(z_t, t, c)$ the denoising network. A scheduler update can be written abstractly as

$$z_{t-1} = S_t\left(z_t, \epsilon_\theta(z_t, t, c), \xi_t\right), \tag{1}$$

where S_t is the numerical scheduler and ξ_t denotes stochastic state when present. Deterministic samplers such as DDIM [14] reduce randomness, but the output remains dependent on scheduler configuration, conditioning, seed, model weights, and intermediate latents.

Image editing methods build on this process in different ways. SDEdit adds noise to an existing image and denoises under new conditioning [15]. Prompt-to-Prompt modifies cross-attention maps to preserve layout while changing prompt semantics [4]. DiffEdit estimates masks for semantic differences [16]. ControlNet adds conditional control signals such as edges or pose [5]. These techniques are valuable, but they do not by themselves define a full workflow model for persistent semantic regions, cached repeated edits, and rollback.

2.3. Semantic annotation for editable layers

Semantic layer extraction can use several sources. For images generated from prompts, cross-attention maps can relate prompt tokens to spatial regions; DAAM provides one method to interpret Stable Diffusion attention [17].

For arbitrary images, open-vocabulary object detection and segmentation are more general. GroundingDINO detects text-specified regions in an image [18], while SAM and SAM2 provide high-quality mask generation from prompts such as boxes or points [19, 20]. Depth models can add foreground/background ordering [21].

Our prototype uses GroundingDINO+SAM2 for the heavy benchmark because it works on generated images without needing access to generation-time attention maps. The broader architecture allows multiple detectors to contribute to a semantic layer ensemble: DAAM for prompt-token memory, GroundingDINO+SAM2 for object masks, face landmarks for facial parts, OCR/layout detectors for text, and depth estimation for layer ordering.

3. Related Work

3.1. Workflow systems for heterogeneous computation

Scientific workflow research has long studied how to describe, schedule, execute, monitor, and reproduce computations composed of many dependent tasks. Systems such as Pegasus show the value of explicit workflow representations, provenance capture, and task planning across heterogeneous resources [7]. Broader surveys identify recurring concerns: abstraction, interoperability, execution monitoring, fault tolerance, data staging, and reproducibility [1, 2]. These concerns are directly relevant to generative AI editing, even though the application domain is different. A diffusion editing session has a workflow graph, expensive tasks, intermediate artifacts, and user-driven branching.

The compute-continuum setting adds another layer of complexity when editing tasks are placed on local accelerators, cloud GPUs, edge devices, or HPC resources. Prior scheduling work for heterogeneous systems, including list scheduling heuristics such as HEFT, shows that task-resource matching can substantially affect makespan [22]. In generative AI workflows, useful scheduling features include not only task duration and dependency depth, but also semantic scope, mask size, cache availability, and quality risk. This paper contributes a domain-specific workflow model that exposes those features, while leaving multi-tier scheduling evaluation to future work.

3.2. Provenance and reproducible AI pipelines

Provenance has been central to scientific workflows because researchers must know how an output was derived, which inputs were used, and which computation produced each artifact [9]. The W3C PROV model formalizes entities, activities, and agents for representing derivations [10]. Generative AI workflows need the same discipline. A generated image is not fully described by its pixels. It depends on model weights, scheduler configuration, prompts, seeds, detector outputs, masks, crop geometry, blending parameters, and previous versions.

The FAIR principles emphasize that scientific data and artifacts should be findable, accessible, interoperable, and reusable [11]. For generative workflows, this requires more than saving final images. It requires publishing workflow code, model identifiers, configuration, seeds, metrics, and enough intermediate artifacts to reproduce or audit the result. The public artifact for this paper includes the manuscript source, benchmark scripts, reports, aggregate metrics, and representative image grids; model checkpoints are not redistributed because they are governed by separate licenses.

3.3. Diffusion editing and semantic control

Diffusion editing methods improve controllability in several ways. SDEdit performs image editing by adding noise to an input and denoising under new guidance [15]. Prompt-to-Prompt preserves layout by controlling cross-attention maps during prompt edits [4]. DiffEdit estimates semantic masks that separate regions implied by prompt differences [16]. InstructPix2Pix trains a model to follow natural-language image editing instructions [6]. ControlNet adds external spatial controls such as edges, depth, and pose [5]. These methods operate mostly at the editing-model level. They answer how to produce an edit, but not how to manage an iterative workflow of many edits with cached semantic state.

Semantic segmentation and open-vocabulary grounding provide another route. SAM and SAM2 provide promptable mask generation [19, 20], while GroundingDINO maps text labels to detection boxes [18]. These methods make it possible to construct named image layers from arbitrary images. However, a detector output alone is not a workflow. The contribution here is to bind detector outputs to provenance, versioning, execution policy, and performance metrics.

3.4. Positioning

The closest existing ideas are semantic image editing, attention-based diffusion control, inpainting, and workflow provenance. Our work differs by making the semantic layer graph the persistent execution object. The graph is not merely a UI selection or a temporary mask; it is a cacheable workflow artifact that drives scheduling, repeated editing, rollback, and reproducibility. This positioning is intentionally systems-oriented. The paper does not propose a new denoising network or segmentation model. It proposes a workflow model for deciding when and how to use existing generative and semantic components.

4. Problem Formulation

4.1. Editing as a semantic workflow

Let an editing session be represented as a directed acyclic workflow

$$W = (T, D, R, P), \tag{2}$$

where T is a set of tasks, D is a set of data dependencies, R is a set of resource requirements, and P is provenance. In

a diffusion editing session, tasks include base generation, semantic annotation, mask refinement, crop selection, local diffusion, deterministic projection, compositing, metric computation, caching, and rollback.

We represent the current image state as a semantic layer graph

$$G = (V, E), \tag{3}$$

where each vertex $v_i \in V$ is a semantic layer and each edge $e_{ij} \in E$ represents spatial, hierarchical, or dependency relationships. A layer is defined as

$$\ell_i = (y_i, \mathbf{M}_i, b_i, d_i, h_i, p_i, c_i), \tag{4}$$

where y_i is the semantic label, $\mathbf{M}_i \in \{0,1\}^{H\times W}$ is the mask, b_i is the bounding box, d_i is optional depth/order metadata, h_i is edit history, p_i is provenance, and c_i is cache state. Hierarchical edges can express relations such as mouth inside face or object in foreground; dependency edges can express likely co-edit regions such as hand–wrist–sleeve or sky–horizon–reflection.

An edit request is

$$e = (q, a, s, \kappa), \tag{5}$$

where q is the target query, a is the action, s is edit strength, and κ encodes user constraints such as quality preference, latency budget, determinism, or preservation requirement. The workflow must resolve q to one or more layers, estimate the effect of a, choose an execution operator, and produce a new version state.

4.2. Execution operators

We define five operators.

Naive full regeneration recomputes the full image from text or image conditioning. It has high global coherence but weak preservation. Its execution cost is approximately the cost of a complete diffusion pass:

$$T_{\text{naive}} \approx \sum_{t=1}^{N} C_\theta(H, W, t), \tag{6}$$

where C_θ is the model cost at resolution $H \times W$.

Semantic local diffusion crops a region around the target mask, runs img2img or inpainting-like diffusion on the crop, and composites the result through the mask:

$$x' = \mathbf{M}_i \odot f_\theta(x_{b_i}, e) + (1 - \mathbf{M}_i) \odot x. \tag{7}$$

The expected cost is

$$T_{\text{local}} \approx T_{\text{det}} + \sum_{t=1}^{N_i} C_\theta(h_i, w_i, t) + T_{\text{comp}}, \tag{8}$$

where T_{det} is detector time, $h_i \times w_i$ is crop size, N_i is the local denoising step count, and T_{comp} is compositing time.

Cached semantic re-edit reuses the existing semantic mask, bounding box, crop policy, and layer provenance. It removes repeated detector cost:

$$T_{\text{cached}} \approx \sum_{t=1}^{N_i} C_\theta(h_i, w_i, t) + T_{\text{comp}}. \tag{9}$$

This operator represents the strongest practical advantage of the workflow model.

Deterministic projection applies a non-diffusion transformation to the layer, such as hue or color projection:

$$x' = \mathbf{M}_i \odot g(x_i, a) + (1 - \mathbf{M}_i) \odot x, \tag{10}$$

where g is deterministic. It is suitable for simple color edits when exact locality and speed matter more than generative realism.

Rollback restores a prior layer or composite state:

$$x' = \text{restore}(v_k, \ell_i), \tag{11}$$

where v_k is a stored version. Rollback uses stored layer state and does not invoke diffusion in the prototype.

4.3. Objective function

The workflow policy chooses an operator o from a candidate set $\mathcal{O}$. The objective below should be read as a cost model for comparing feasible operators, not as a claim that the current prototype learns the weights automatically. The prototype uses explicit rules and measured operator costs; the same state variables can later support an optimizer or learned scheduler. We write the normalized operator score as

$$J(o \mid e, \ell_i, C) = \left[\alpha \widehat{T}_o + \beta \widehat{D}_o + \gamma \widehat{Q}_o + \delta \widehat{U}_o + \eta \widehat{R}_o\right], \tag{12}$$

and the selected operator is

$$\pi(e, \ell_i, C) = \arg \min_{o \in \mathcal{O}_{\text{feasible}}} J(o \mid e, \ell_i, C). \tag{13}$$

Here $\widehat{T}_o$ is normalized latency, $\widehat{D}_o$ is outside-target drift, $\widehat{Q}_o$ is expected quality or artifact risk, $\widehat{U}_o$ is user effort, and $\widehat{R}_o$ is resource cost. If watt telemetry or cloud billing is available, $\widehat{R}_o$ can be instantiated as normalized energy or monetary cost. In the present benchmark, no power draw is measured, so $\widehat{R}_o$ is instantiated only as a compute-work proxy:

$$\widehat{R}_o = \frac{N_o A_o}{N_{\text{full}} HW} + \chi_o \frac{T_{\text{det}}}{\bar{T}_{\text{det}}}, \tag{14}$$

where N_o is the number of denoising steps, A_o is the rendered pixel area, $H \times W$ is the full image area, and $\chi_o = 1$ when the operator invokes detector inference. This proxy does not claim to be a joule measurement; it records the relative amount of diffusion work and detector work exposed to the scheduler. C denotes context such as available cache, model family, accelerator state, privacy preference, and deadline.

Outside-target drift is estimated as

$$D(x, x', \mathbf{M}) = \frac{1}{|\Omega_{\bar{\mathbf{M}}}|} \sum_{p \in \Omega_{\bar{\mathbf{M}}}} \mathbb{I}\left(\|x_p - x'_p\|_1 > \tau\right), \tag{15}$$

where $\Omega_{\bar{\mathbf{M}}}$ is the non-target pixel set and τ is a pixel-change threshold. Inside-target change is

$$I(x, x', \mathbf{M}) = \frac{1}{|\Omega_{\mathbf{M}}|} \sum_{p \in \Omega_{\mathbf{M}}} \|x_p - x'_p\|_1. \tag{16}$$

Boundary risk is measured on a dilated mask band:

$$B = \mathrm{dilate}(\mathbf{M}, r) - \mathrm{erode}(\mathbf{M}, r), \tag{17}$$

and evaluated using the same pixel-change statistic. Boundary-band change is introduced here as a proxy for structural blending and seam artifacts. It is not necessarily bad; it often indicates that color, antialiasing, shadow, or shading changed near the seam. However, high boundary changes can signal visible artifacts and should be interpreted with visual inspection or perceptual metrics.

4.4. Amortized reuse model

The value of semantic annotation depends on session length. A first edit pays detector cost, but repeated edits reuse the same layer state. Let k be the number of edits applied to the same semantic layer in a session. A naive session cost is approximated as

$$T_{\mathrm{naive}}(k) = T_{n,1} + (k-1)T_{n,2}, \tag{18}$$

where $T_{n,1}$ is the first naive edit and $T_{n,2}$ is a subsequent naive variant. A semantic session cost is

$$T_{\mathrm{sem}}(k) = T_{\mathrm{det}} + T_{s,1} + (k-1)T_{s,c}, \tag{19}$$

where $T_{s,1}$ is the first local render and $T_{s,c}$ is a cached re-edit. Semantic execution is beneficial when

$$T_{\mathrm{det}} + T_{s,1} + (k-1)T_{s,c} < T_{n,1} + (k-1)T_{n,2}. \tag{20}$$

Solving for k gives an approximate break-even condition:

$$k > 1 + \frac{T_{\mathrm{det}} + T_{s,1} - T_{n,1}}{T_{n,2} - T_{s,c}}. \tag{21}$$

Using the measured benchmark means $T_{n,1} = 35.546$ s, $T_{n,2} = 35.741$ s, $T_{\mathrm{det}} = 13.543$ s, $T_{s,1} = 30.132$ s, and $T_{s,c} = 8.115$ s. The semantic path is slower for $k = 1$, but faster already at $k = 2$. This is the mathematical reason that repeated edits and version exploration are the strongest use case.

4.5. Layer dependency risk

Not all local edits are independent. A command such as "change the hand" may affect fingers, wrist, sleeve, object contact, shadow, and reflection. We model this with a dependency expansion function

$$\Gamma(\ell_i, a) \subseteq V, \tag{22}$$

which returns layers that may need protection or co-editing for action a. The effective edit support is then

$$\mathbf{M}_{\mathrm{eff}} = \bigcup_{\ell_j \in \Gamma(\ell_i, a)} w_j \mathbf{M}_j, \tag{23}$$

where w_j can encode soft dependency weights. A concrete graph construction can combine geometry, attention, depth, and contact evidence:

$$s_{ij} = \lambda_m \mathrm{IoU}(\mathbf{M}_i, \mathbf{M}_j) + \lambda_a O(A_i, A_j) + \lambda_z Z(d_i, d_j) + \lambda_c C(\partial\mathbf{M}_i, \partial\mathbf{M}_j), \tag{24}$$

with an edge $(i, j) \in E$ when $s_{ij} > \theta_E$. $O(A_i, A_j)$ measures overlap between token-attention maps, for example from DAAM; $Z(d_i, d_j)$ encodes depth-order or occlusion compatibility; and $C(\partial\mathbf{M}_i, \partial\mathbf{M}_j)$ captures boundary contact or near-contact. The current prototype implements only the geometry part of this idea through crop padding and boundary bands, because the benchmark uses post-generation GroundingDINO+SAM2 masks rather than stored attention maps. The formulation makes the missing dependency edges explicit and shows how a later implementation can build E programmatically instead of relying on a fixed padding heuristic. Dependency expansion changes crop size, latency, and expected quality risk, so it is relevant to any scheduler that uses layer geometry.

4.6. Determinism and reproducibility

Diffusion pipelines are only conditionally deterministic. A run is reproducible when the model checkpoint, VAE, scheduler, timestep sequence, seed, prompt, negative prompt, guidance scale, precision, image size, backend, and software versions are fixed. Even then, hardware kernels and mixed precision can introduce small numerical differences. Our workflow model treats determinism as a provenance property rather than an assumption. Each artifact records enough metadata to make repeated execution auditable, and deterministic projection/rollback are separated from stochastic or numerically sensitive diffusion operators.

5. System Architecture

Figure 1 shows the prototype architecture. A request enters the system as either a base generation prompt or an edit to an existing image. Base generation produces the initial image and provenance state. Semantic annotation then creates named candidate layers using GroundingDINO+SAM2 in the heavy benchmark. The resulting

Image DOM stores masks, boxes, labels, crop policies, version identifiers, and provenance. The policy engine selects an execution operator. The execution layer runs full diffusion, local diffusion, deterministic projection, cached re-edit, or rollback. Finally, the compositor writes a new version and logs metrics.

5.1. Semantic Image DOM

The Image DOM is the persistent state that makes repeated edits efficient. For each layer, it stores the mask, bounding box, label, detector backend, detector confidence where available, crop geometry, parent/child relations, and edit history. It also stores global provenance: model checkpoint name, scheduler, inference steps, seed, image size, guidance scale, and prompt. The representation is intentionally close to workflow provenance models: it records entities, activities, and derivations rather than only final pixels [10].

The current prototype uses file-system persistence for simplicity. Each sample output folder contains the base image, detector mask overlay, naive edit outputs, semantic local outputs, projection-only output, rollback output, sample grid, and a metrics record. This is sufficient for reproducible experiments. A production implementation would replace this with a content-addressed store or workflow database.

5.2. Detector and mask stage

The heavy benchmark uses GroundingDINO for open-vocabulary target localization and SAM2 for mask generation. The target labels are selected from the edit category: lips for face edits, shirt for body edits, mug for object edits, and sky for landscape edits. The detector stage returns a mask and bounding box. The local renderer expands the bounding box with padding and snaps to a crop size supported by the model backend.

Detector time is included as a separate metric because semantic annotation is expensive enough to change the conclusion. A one-off edit with no cached layer state may be slower than naive generation, while a repeated edit on the same layer avoids detection and becomes much faster.

5.3. Execution policy

The prototype policy is rule-based but grounded in the cost model above. Projection-only is selected for deterministic color changes when the target layer is already known. Semantic local diffusion is selected when the edit requires generative texture or lighting. Cached re-edit is selected when the layer mask and crop state already exist. Naive regeneration is used as the baseline and remains appropriate for global, structural, or highly entangled edits.

Algorithm 1: Rule-based semantic workflow policy.

1. Input edit $e = (q, a, s, \kappa)$, candidate layers V, context C, and version state.
2. Resolve q to a target layer ℓ_i, or run detector inference if no matching layer exists.
3. Construct feasible operators O_{feasible}: rollback, projection, cached local edit, first local edit, and full regeneration.
4. Remove operators that violate hard constraints, such as unavailable cache, privacy restrictions, unsupported model family, or missing mask.
5. Estimate $\widehat{T}_o$, $\widehat{D}_o$, $\widehat{Q}_o$, $\widehat{U}_o$, and $\widehat{R}_o$ from layer geometry, cache state, detector status, crop size, and benchmarked model family.
6. Prefer rollback for undo, projection for simple deterministic color edits, cached local editing for repeated local variants, first local editing when preservation is required, and full regeneration for global or highly entangled edits.
7. If more than one operator remains plausible, return the feasible operator with the lowest $J(o \mid e, \ell_i, C)$.

Figure 2 gives a conceptual view of the operator space. The boundary between regions is not fixed; it depends on detector availability, cache state, latency budget, crop size, and quality requirements. This is why we report both generation-only and detector-inclusive speedups.

5.4. Implementation and artifact layout

The prototype is implemented as Python scripts around local Diffusers pipelines and local model checkpoints. The heavy benchmark driver creates a resumable output directory and appends a JSON record after every completed sample. This design was important in practice because SDXL model transitions can fragment MPS memory; a resumed process can skip completed records and continue from the next missing model/case/sample tuple.

Each sample directory follows the same structure:

```
sample_XX/
  base.png
  mask_overlay.png
  naive_v1.png
  naive_v2.png
  semantic_raw_v1.png
  semantic_projected_v1.png
  semantic_cached_v2.png
  projection_only.png
  rollback.png
  sample_grid.png
  metrics.json
```

The exact filenames vary slightly by operator, but the invariant is that every image artifact is paired with a metrics record. The aggregate directory contains a full JSON file, a Markdown report, a model-case CSV matrix, and representative sample grids. This layout is intentionally simple: it can be inspected without a database, archived with the paper, or converted into a formal workflow provenance store.

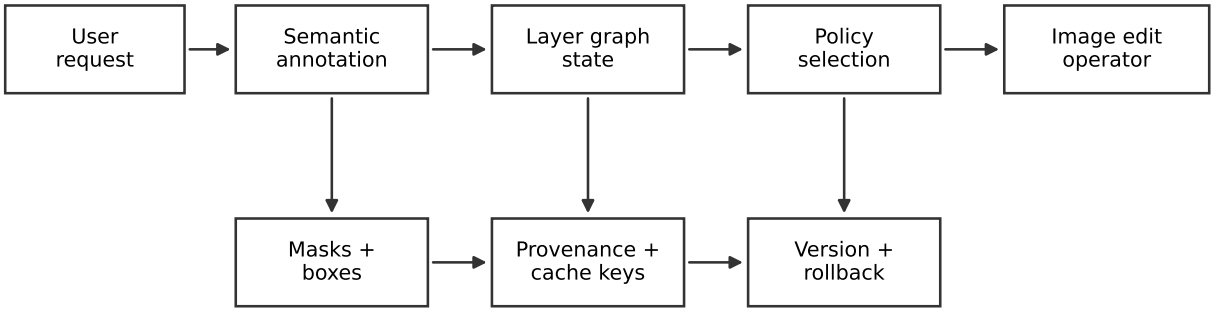

Figure 1: Semantic layer workflow architecture. The key difference from a monolithic diffusion call is that semantic annotation, layer graph state, cache keys, and version provenance are first-class workflow artifacts.

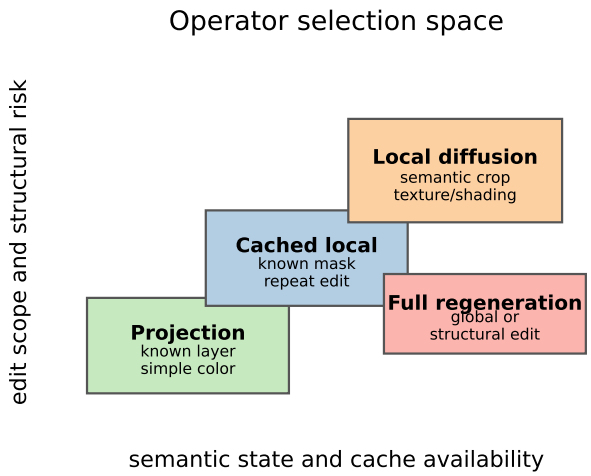

Figure 2: Conceptual operator selection space. Simple local edits favor deterministic projection or cached local rendering, while structurally broad edits may require full regeneration.

The policy engine is therefore a transparent heuristic scheduler rather than an optimized solver. This is a deliberate prototype choice: the experiment first tests whether the Image DOM exposes useful scheduling features. The parameters $\alpha, \beta, \gamma, \delta, \eta$ can later be fitted from user preferences, measured energy, or deployment constraints, but the current paper reports the rule-based policy and the cost-model simulations separately.

5.5. Failure handling

Long-running generative workflows benefit from partial-failure tolerance. The benchmark treats each sample tuple as an idempotent unit identified by model key, case key, and sample index. If execution stops after writing a record, the next run skips that tuple. This allowed the local benchmark to retain progress after memory-related restarts.

6. Experimental Methodology

6.1. Research questions

The evaluation is organized around six questions:

RQ1 Does semantic workflow execution reduce non-target drift compared with naive full-image regeneration?

RQ2 When detector overhead is excluded, does local semantic rendering reduce generation time?

RQ3 What is the first-edit latency when semantic detection is included?

RQ4 How much speedup is obtained from cached semantic re-edits and versioning?

RQ5 How does performance vary by model family and edit category?

RQ6 What trade-offs appear between deterministic projection, local diffusion, and full regeneration?

RQ7 How sensitive are scheduling conclusions to heterogeneous resource assumptions?

6.2. Benchmark design

We ran a paired benchmark over 10 compatible local diffusion checkpoints, four edit categories, and five seed variants per category, for 200 paired samples. Each sample executes the same workflow pattern:

1. Generate or load a base image at 768×768.
2. Execute a naive full-image first edit.

3. Execute a second naive variant edit.
4. Detect the target semantic layer using GroundingDINO and SAM2.
5. Execute projection-only deterministic editing.
6. Execute semantic local diffusion on the target crop and composite through the mask.
7. Execute a cached semantic re-edit using the same layer state.
8. Execute rollback to a prior version.
9. Save images, masks, sample grids, and metrics.

All runs use 20 denoising steps. The four cases are face/lips color editing, human body/shirt editing, object/mug editing, and landscape/sky editing. These categories were chosen because they cover small facial parts, medium human regions, discrete objects, and broader background regions. The local rendering crop is determined from the SAM2 mask and may be smaller than the full image, but it is padded and quantized for model compatibility.

The benchmark ran on the local workstation using the available Apple MPS backend. Some SDXL model transitions required process restart because of MPS memory fragmentation; the benchmark is resumable and persisted completed records were reused rather than recomputed. This is reported as part of the realistic local execution environment, not hidden as a failure. The final benchmark completed 200 samples with zero per-sample errors. Four incompatible local checkpoint files were excluded because they were inpainting-only, SD3, non-image-generation, or incomplete checkpoint entries.

6.3. Models

Table 1 lists the 10 compatible checkpoints. The set intentionally includes both SD1.5-family and SDXL-family models because throughput and img2img cost differ substantially across families.

6.4. Metrics

The benchmark records four groups of metrics.

Latency metrics include naive first-edit time, naive second-edit time, detector time, semantic local render time, cached semantic re-edit time, projection-only time, and rollback time. We report three speedup variants:

$$S_{\text{gen}} = T_{\text{naive}}/T_{\text{semantic}}, \tag{25}$$

$$S_{\text{incl}} = T_{\text{naive}}/(T_{\text{det}} + T_{\text{semantic}}), \tag{26}$$

$$S_{\text{cached}} = T_{\text{naive2}}/T_{\text{cached}}. \tag{27}$$

S_{gen} asks whether local rendering is faster than full rendering once a layer is known. S_{incl} asks whether the first semantic edit is faster for a user with no cache. S_{cached} asks whether repeated edits benefit from the persisted Image DOM.

Locality metrics measure outside-target changed pixels, inside-target mean absolute difference, and boundary-band changed pixels. These quantify preservation and seam risk. The outside-target metric is especially important because full-image regeneration can be visually plausible while still destroying identity or layout.

Layer geometry metrics include mask area percentage, bounding-box area percentage, crop size, and crop-pixel percentage. These help explain why some local edits are faster than others.

Versioning metrics include cached re-edit time and rollback time. These directly test whether semantic state makes repeated edits and undo operations cheap.

Resource proxy metrics include crop-pixel percentage and detector invocation. These are not watt-level energy measurements. They instantiate the resource term $\widehat{R}_o$ as relative diffusion area and detector work so that the scheduler can compare full rendering, local rendering, projection, and rollback on a common scale.

6.5. Reproducibility artifacts

The benchmark stores machine-readable results in JSON and CSV. The JSON contains all per-sample records and aggregate summaries. The CSV contains the model-by-case matrix used for compact comparison. Figures in this manuscript are generated from those files rather than manually transcribed. The manuscript directory also includes copied sample grids for the appendix. The most important reproducibility fields are model key, model family, case key, sample index, seed, image width/height, steps, mask area, bounding-box area, crop size, detector time, render times, speedups, drift metrics, and artifact directory.

The public artifact does not include model checkpoints or detector weights because those files are large and governed by separate licenses. It includes the scripts, reports, aggregate metrics, model-case matrix, manuscript source, and representative image artifacts needed to inspect the reported results.

7. Results

7.1. Overall performance

Table 2 summarizes the full 200-sample benchmark. Semantic local rendering excluding detector cost is faster than naive full-image generation on average. However, detector-inclusive first edits are slower on average. Cached re-edits are the strongest performance result, with a 7.041x average speedup. Projection-only edits are effectively instantaneous for simple color changes, and rollback is near-interactive.

The detector-inclusive number is the first-edit result. When a user edits a region for the first time and the system has no cached semantic layer, GroundingDINO+SAM2

Table 1: Evaluated diffusion checkpoints. Each model is evaluated on four edit categories with five seed variants per category.

Model	Family	Samples	Naive (s)	Semantic local (s)	Detector (s)
Deliberate v6	SD1.5	20	12.213	8.180	12.954
DreamShaper 8	SD1.5	20	31.388	8.298	13.292
DreamShaper XL 1.0	SDXL	20	39.873	55.469	14.677
Juggernaut Reborn	SD1.5	20	11.390	4.973	12.778
Juggernaut XL v2	SDXL	20	118.220	75.622	15.298
OpenJourney v4	SD1.5	20	23.941	7.496	12.384
RealVisXL V4.0	SDXL	20	53.656	62.119	14.488
Realistic Vision V6.0 B1	SD1.5	20	10.540	9.139	12.194
SDXL Base 1.0	SDXL	20	43.568	60.456	14.151
Stable Diffusion 1.5 Base	SD1.5	20	10.675	9.567	13.220

Table 2: Overall benchmark results across 200 paired executions.

Metric	Mean
Naive full edit time	35.546 s
Semantic local edit time, detector excluded	30.132 s
Detector time	13.543 s
Projection-only edit time	0.031 s
Generation-only semantic speedup	1.788x
One-off detector-inclusive speedup	0.809x
Cached version re-edit speedup	7.041x
Projection-only speedup	1227.276x
Rollback time	0.188 s
Naive outside-target changed pixels	93.954%
Semantic outside-target changed pixels	0.000%
Semantic boundary-band changed pixels	42.301%
Mean crop-pixel work proxy	42.145%
Mean mask area	6.032%

Table 3: Operator-level baseline decomposition across 200 samples.

Operator	Mean time
Naive full regeneration	35.546 s
Detector + local semantic edit	43.675 s
Known-mask local rendering	30.132 s
Cached semantic re-edit	8.115 s
Deterministic projection	0.031 s
Rollback	0.188 s

adds substantial overhead. This overhead can be justified when preservation matters, but it does not guarantee lower latency. Once the semantic layer exists, repeated variants avoid detection and reuse the layer state.

7.2. Baseline decomposition

Table 3 separates the benchmark into operator-level baselines. This comparison is important because the workflow is not only evaluated against full-image regeneration. The detector-inclusive local edit represents an ordinary first local edit with semantic detection but no reuse. Known-mask local rendering removes detector cost. Cached re-editing then measures the additional value of persisted workflow state.

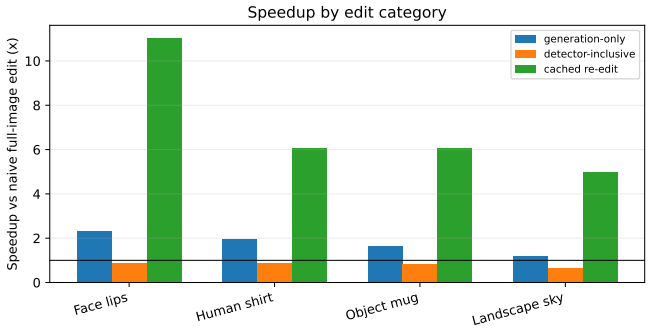

Figure 3: Speedup by edit category. Cached semantic re-edit is positive for all categories, while detector-inclusive first-edit speedup is below 1x on average.

The decomposition shows why caching is central to the contribution. The first detector-inclusive local edit is slower than full regeneration on average, but known-mask local rendering is faster, and cached semantic re-editing is 5.382x faster than the detector-inclusive local baseline. The workflow gain therefore comes from making semantic state persistent and reusable, not from assuming that every local edit is automatically faster than a full-image edit.

7.3. Edit category effects

Table 4 and Figure 3 show results by edit category. Face/lips edits have the highest cached speedup at 11.055x because the editable region is small and repeated color changes are cheap. Human body/shirt and object/mug edits show similar cached speedups near 6x. Landscape/sky edits are less favorable because the semantic region and crop are often larger, but cached re-editing still averages 4.966x.

These results support a policy-based interpretation. If the target is small and the edit is simple, projection-only or cached local editing is attractive. If the target is broad, first-edit performance may not improve, but preservation can still justify semantic execution. Layer geometry and cache state are therefore useful scheduling features.

Table 4: Results by edit category. Each category contains 50 paired samples.

Case	Naive	Local	Det.	Gen.	Incl.	Cached	Naive drift	Sem. drift
Face lips	35.796	28.063	13.871	2.313x	0.877x	11.055x	90.355%	0.000%
Human body shirt	38.372	30.619	13.517	1.958x	0.886x	6.085x	91.908%	0.000%
Landscape sky	31.858	31.358	13.636	1.211x	0.634x	4.966x	98.745%	0.000%
Object mug	36.160	30.488	13.151	1.669x	0.838x	6.058x	94.809%	0.000%

7.4. Model effects

Figure 4 shows strong model dependence. DreamShaper 8, OpenJourney v4, and Juggernaut XL v2 achieve detector-inclusive first-edit speedups above 1x. Other models, especially several SDXL checkpoints, are slower in the semantic first-edit path. This occurs because local img2img throughput and full-generation throughput differ by checkpoint and pipeline implementation. It would be misleading to claim that local semantic rendering is always faster.

The workflow conclusion remains stable across models: cached re-editing is positive for every checkpoint. The weakest cached result is RealVisXL V4.0 at 3.172x, while the strongest is OpenJourney v4 at 12.229x. This suggests that semantic caching is a robust systems mechanism even when first-edit acceleration is model-dependent.

7.5. Locality and drift

Naive full-image regeneration changes an average of 93.954% of outside-target pixels. Semantic compositing changes 0.000% outside-target pixels under the exact pixel-change metric because the edited crop is composited through the semantic mask and the unmasked base image is preserved.

This metric is intentionally strict. It does not claim that semantic edits are always perceptually superior; it shows that the workflow preserves non-target pixels by construction. This is useful when identity, layout, text, object positions, or scientific visual state must remain stable. The boundary-band change average of 42.301% indicates that changes occur near seams, so visual seam quality still requires inspection and better blending methods.

7.6. Trade-offs

The main trade-off is between first-edit latency and state reuse. Semantic editing pays an annotation cost before it can exploit locality. In the benchmark, this cost is large enough that the first detector-inclusive edit is slower than naive regeneration on average. This is acceptable when the user expects to make several changes to the same region, compare variants, or preserve the surrounding image exactly. It is less attractive for a single low-stakes edit where global regeneration is visually acceptable and latency is the only concern.

The second trade-off is between preservation and visual integration. Mask compositing keeps non-target pixels fixed, which is useful for identity, layout, text, product images, or scientific visual state. The same constraint can make boundaries more visible if the local edit changes lighting, texture, or geometry near the mask edge. In those cases, semantic execution is still useful as a controlled editing mode, but it should be paired with better blending, boundary-aware scoring, or a policy that expands the crop when the edit is structurally coupled to nearby regions.

A third trade-off is implementation complexity. Maintaining masks, cache keys, provenance, and version state is more work than calling a diffusion pipeline once. The benefit appears when edits are iterative, audited, or reused: cached re-edits become faster, rollback is cheap, and the system can explain which model, mask, crop, and parameters produced each result. For casual one-shot generation this overhead may not be justified; for repeated editing workflows it becomes the mechanism that makes locality and reproducibility practical.

7.7. Versioning and rollback

Rollback averages 0.188 s. This supports the versioning model: undoing a semantic edit restores cached state rather than invoking diffusion. In interactive workflows, rollback changes the editing process from destructive regeneration to a versioned exploration tree. Users can compare variants of the same object while preserving the rest of the image and retaining intermediate states.

The cached re-edit result is similarly important. A first semantic edit may be slower because of detector overhead, but subsequent edits amortize semantic annotation. This is analogous to scientific workflows where expensive preprocessing or indexing steps become worthwhile when reused across many downstream tasks.

7.8. Scheduling-policy simulation

Table 5 applies a simple scheduling simulation to the 200 measured records. For a session with k edits on the same layer, the naive policy uses full regeneration for every variant, the semantic policy pays detector cost once and then uses cached re-edits, and the oracle latency policy chooses the faster of the two measured paths per sample. This is not a new deployment experiment, but it evaluates the scheduling decision that the workflow representation exposes.

The policy result is stronger than the mean break-even calculation because it shows how often each decision is preferred across heterogeneous models and edit categories. For $k = 1$, semantic editing is not a latency default. For $k = 2$, reuse already changes the decision for most samples. For $k = 5$, semantic execution is faster almost everywhere. This supports the systems claim that session context, cache state, and preservation constraints should

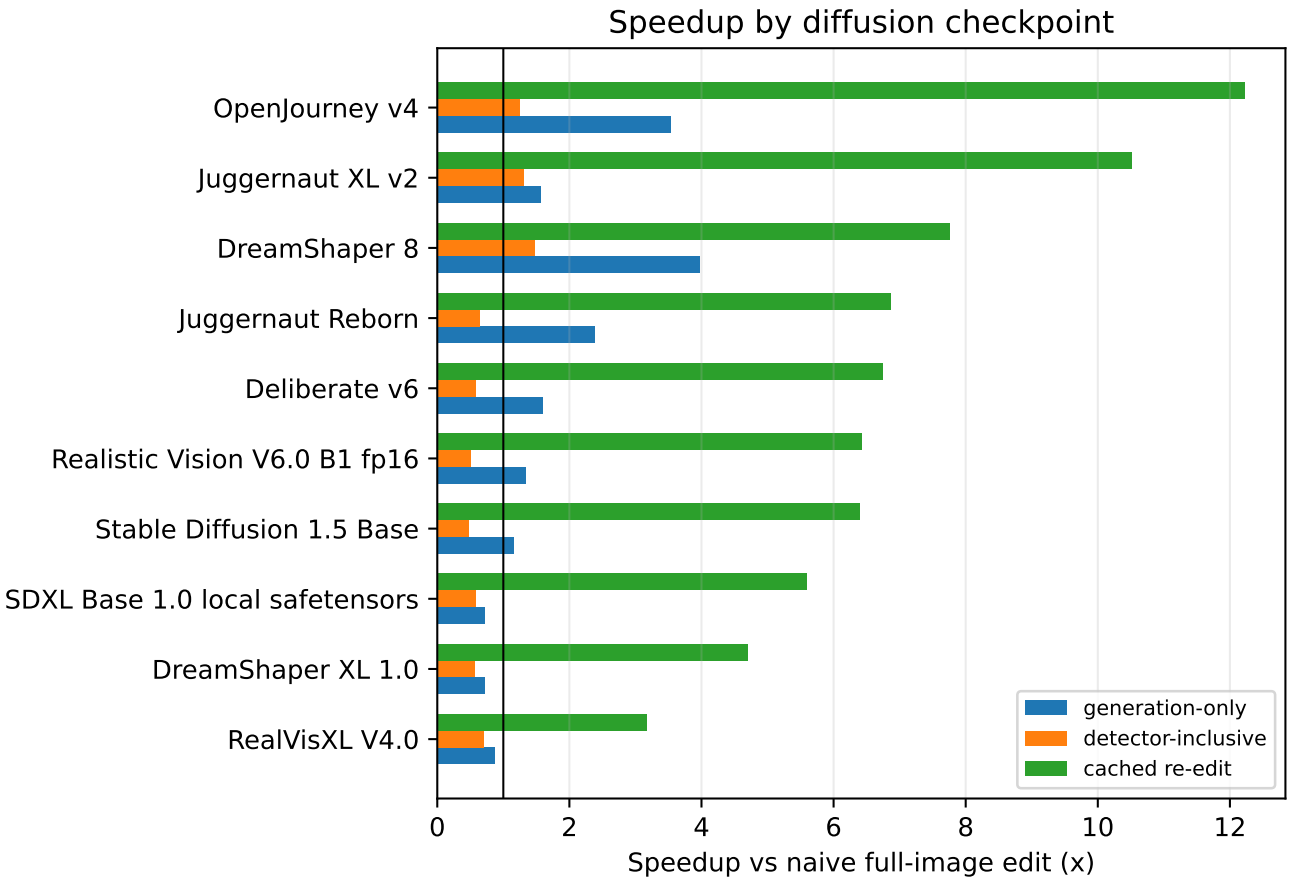

Figure 4: Speedup by model. The generation-only metric isolates local rendering cost; the detector-inclusive metric represents first-edit user latency; cached re-edit represents repeated semantic layer editing.

Table 5: Measured scheduling-policy simulation. k is the number of edits on the same semantic layer. Times are mean seconds across 200 samples.

k	Naive	Semantic	Oracle	Sem. wins
1	35.546	43.675	30.780	25.5%
2	71.287	51.791	49.543	60.0%
3	107.027	59.906	59.839	98.0%
5	178.509	76.137	75.967	99.5%

be explicit scheduler inputs rather than hidden inside a single image-generation call.

7.9. Heterogeneous resource sensitivity

The measured benchmark uses Apple MPS, so the absolute detector and diffusion times should not be treated as universal. To test whether the scheduling conclusion depends only on this desktop configuration, Table 6 applies a sensitivity model to the 200 measured records. The model scales detector time and diffusion time separately:

$$T^r_{\text{naive}} = s^r_{\text{diff}} T_{\text{naive}} + L^r, \tag{28}$$

$$T^r_{\text{sem}} = s^r_{\text{det}} T_{\text{det}} + s^r_{\text{diff}} T_{\text{local}} + L^r, \tag{29}$$

$$T^r_{\text{cached}} = s^r_{\text{diff}} T_{\text{cached}} + L^r, \tag{30}$$

where s^r_{det} and s^r_{diff} are resource-tier scaling factors and L^r is a transfer or orchestration penalty. The factors are not presented as measurements of named commercial hardware; they are stress-test scenarios that represent slower edge execution, the measured local run, faster CUDA-class execution, and a cloud accelerator with transfer overhead.

The projection preserves the main conclusion but changes its magnitude. Faster accelerators reduce both full-generation and local-diffusion latency, while faster detectors narrow the first-edit penalty. In all four scenarios, the one-off semantic path remains slower on average, but the three-edit session is faster because detection is amortized and cached re-edits are cheap. This is the compute-continuum claim supported by the present evidence: the exact break-even point is resource dependent, but the Image DOM exposes the variables a placement scheduler needs to recompute the decision.

7.10. Interpreting boundary-band change

The average semantic boundary-band change is 42.301%. This number should not be interpreted as a failure rate. The boundary band contains pixels close to the mask edge, where color, lighting, and antialiasing changes are expected after compositing. A high value

Table 6: Heterogeneous resource sensitivity projection using the 200 measured records. Times are projected mean seconds. $k = 3$ denotes three edits on the same semantic layer.

Scenario	s_{det}	s_{diff}	Naive $k = 1$	Semantic $k = 1$	Semantic speed	$k = 3$ speed
Edge-constrained	1.80	1.50	53.320	69.576	0.766x	1.709x
Measured Apple MPS	1.00	1.00	35.546	43.675	0.814x	1.787x
CUDA workstation	0.35	0.45	15.996	18.300	0.874x	1.881x
Cloud accelerator, $L = 1.5$ s	0.15	0.20	8.609	9.558	0.901x	1.639x

Table 7: Automatic quality-risk audit across 200 samples. Thresholds are used only as triage flags.

Metric	Mean	Median	Flagged
Detector selection score	0.456	0.473	10.5% below 0.25
Inside-target change	34.435	34.459	3.5% below 20
Boundary-band change	42.301	42.825	3.0% above 50

Table 8: Perceptual validation layer for semantic editing quality.

Metric	Tests	Policy use
CLIP/BLIP alignment	prompt/edit intent	semantic success risk
LPIPS target distance	perceptual change	over/under-edit risk
LPIPS outside mask	perceptual preservation	identity/layout risk
Human seam rating	boundary artifacts	expand crop/blend
Human task rating	edit satisfaction	policy training label

becomes concerning only when the visual seam is obvious. The benchmark therefore treats boundary change as a seam-risk proxy. It is useful for comparing policies and for flagging samples for human inspection, but it is not by itself a perceptual quality score.

7.11. Automatic quality-risk audit

Table 7 adds an automatic quality-risk audit over the 200 samples. The audit does not replace human evaluation; it checks whether the pipeline produced a plausible target, changed the target region, and avoided unusually high boundary disturbance. Detector selection score is used as a target-confidence proxy, inside-target mean absolute difference as an edit-activity proxy, and boundary-band change as a seam-risk proxy.

The audit suggests that most runs produced measurable target-region changes, while a smaller subset should be inspected for weak detection or seam risk. The strongest unresolved quality issue is still semantic correctness: detector confidence and pixel-change metrics cannot prove that the edit satisfied the user's intent. They do, however, provide a reproducible triage layer that can be combined with human or model-assisted perceptual scoring.

7.12. Perceptual validation layer

Pixel metrics are insufficient for final quality assessment in generative editing. The revised evaluation therefore separates first-stage systems metrics from a second-stage perceptual validation layer. The current artifact contains the images, prompts, masks, and per-sample metadata needed to compute these metrics, but the present paper does not claim new CLIP, LPIPS, or human-study measurements. Instead, Table 8 defines the validation layer that should accompany a production scheduler or a follow-up benchmark.

These metrics would change the quality term $\widehat{Q}_o$. For example, a local edit with low outside drift but poor CLIP alignment should not be considered successful, and an edit with low boundary-band change but a visible seam should trigger a larger crop, feathered blending, or full regeneration. Thus, the pixel audit is best understood as a cheap screening layer; semantic alignment and perceptual realism require either model-assisted scoring or human labels.

7.13. Implications for scheduling

The results suggest three scheduling rules, formalized in Algorithm 1. First, if no cache exists and latency is the only objective, the scheduler needs detector-inclusive cost before choosing semantic editing. Second, if preservation or reproducibility is important, semantic editing may be selected even when it is slower. Third, if a layer is likely to be edited more than once, the detector cost is amortized rapidly. The scheduling context is therefore not only the next edit, but also whether the edit belongs to a longer exploratory session.

In a compute-continuum deployment, these rules could guide placement. Detector-heavy first edits might benefit from faster accelerators when privacy and cost allow. Cached projection or rollback can remain local. Large SDXL local diffusion could be offloaded when local memory pressure is high. The Image DOM exposes semantic scope and cache state before execution, which are the features a placement policy would need.

8. Discussion

8.1. What the system is good at

The benchmark identifies four strong use cases. First, semantic layers preserve non-target pixels under the implemented compositing metric. This matters whenever repeated generation would destroy identity, layout, or previously approved regions. Second, cached layer state accelerates repeated edits, which matches real interactive behavior better than one-shot benchmarks. Third, deterministic projection is extremely fast for simple color

changes. Fourth, versioning and rollback become cheap because they are represented as workflow state rather than as new generation tasks.

The system is therefore best described as a workflow optimization layer for generative AI rather than as a replacement for diffusion editing algorithms. It can invoke diffusion when realism is needed, deterministic math when locality and speed dominate, and rollback when the user wants to restore an earlier state.

8.2. What the system is not yet good at

The weakest result is detector-inclusive first-edit latency. GroundingDINO+SAM2 adds about 13.543 s on average in this local setup. If a user performs only one edit and does not care about preserving the rest of the image, naive regeneration may be faster. Some SDXL checkpoints also show slower local img2img execution than full naive generation. These cases identify conditions that the policy must handle.

Mask correctness is another limitation. The current benchmark measures preservation relative to detector masks, not hand-labeled ground truth. A perfect outside-target metric is only meaningful if the mask corresponds to the intended semantic region. Stronger semantic annotation claims would require hand-labeled masks or at least a smaller expert-labeled validation set.

8.3. Compute-continuum implications

The prototype benchmark runs locally, so the measured results should not be read as a deployed multi-tier continuum experiment. Table 6 partially addresses this by projecting the measured operator traces under separate detector, diffusion, and transfer scaling assumptions. The model can still inform continuum scheduling because different workflow tasks have different placement requirements. Projection and rollback can run on local CPU. Detector inference may run locally if privacy matters or remotely if a faster GPU is available. SDXL local diffusion may be placed on a cloud GPU when the local device is memory constrained. These are sensitivity results and design implications rather than measured deployment results.

The policy objective can include resource cost and deadline:

$$(o^*, r^*) = \arg\min_{o,r} \alpha \widehat{T}_{o,r} + \beta \widehat{D}_o + \gamma \widehat{Q}_o + \eta \widehat{R}_{o,r} + \zeta \widehat{M}_{o,r}, \quad (31)$$

where r is a resource tier and $\widehat{M}_{o,r}$ is normalized memory risk. This turns semantic editing into a resource-aware workflow scheduling problem comparable to heterogeneous task scheduling [22]. The layer graph provides scheduling hints that a flat prompt does not: crop size, cache presence, detector availability, and dependency risk.

8.4. FAIR and reproducibility

The benchmark stores seeds, model identifiers, prompts, detector outputs, masks, images, metrics, and reports. This supports findability and reusability at the experiment level. A stronger FAIR package would include a containerized environment, model hashes, exact dependency lockfiles, and a public dataset or reproducibility bundle. The workflow model is compatible with this direction because provenance is part of the core state.

9. Research Agenda Toward a Compute-Continuum Workflow System

The prototype supports the basic semantic workflow claim, while also showing that a local benchmark is not enough for a full compute-continuum system. This section outlines the next research steps and how they follow from the current results.

9.1. Latent trajectory reuse

The current implementation caches image-space layers, masks, crops, and version artifacts. It does not yet cache the full diffusion latent trajectory. A stronger system would store

$$\mathcal{T}^0 = \{z_N^0, z_{N-1}^0, \ldots, z_0^0\}, \quad (32)$$

where z_t^0 is the base latent at denoising step t. An edit could then restart from an intermediate timestep t_r rather than from pure noise or from the fully decoded image. The edited trajectory would be constrained by the semantic mask:

$$z_{t-1}^e = \mathbf{M}^z \odot S_t(z_t^e, c_e) + (1 - \mathbf{M}^z) \odot z_{t-1}^0, \quad (33)$$

where $\mathbf{M}^z$ is the semantic mask downsampled to latent resolution and c_e is the edit conditioning. This projection preserves the original trajectory outside the edit support while allowing the target region to denoise under new conditioning.

This is not the same as reusing whole denoising steps. A diffusion step is global: changing one region can change the model's prediction elsewhere. The valid reuse object is the cached latent field and the valid constraint is regional projection. The workflow policy would choose t_r from edit strength, dependency risk, and quality target:

$$t_r = \rho(a, s, \Gamma(\ell_i, a), q_{\min}), \quad (34)$$

where a is action type, s is strength, Γ is dependency expansion, and $q_{\min}$ is minimum expected quality. Color edits may restart late or use deterministic projection; structural edits may restart earlier. This is the natural mathematical extension of the current layer cache.

9.2. Continuum-aware placement

A continuum version of the system can treat each operator as a task that may run on multiple resource tiers. Let $R = \{r_1, \ldots, r_m\}$ be candidate resources such as local CPU, local accelerator, edge GPU, cloud GPU, or HPC batch node. Each operator-resource pair has an estimated tuple

$$\phi(o, r) = (T_{o,r}, E_{o,r}, C_{o,r}, M_{o,r}, P_{o,r}), \quad (35)$$

where T is time, E energy, C monetary cost, M memory risk, and P privacy risk. Placement becomes

$$(o^*, r^*) = \arg\min_{o,r} \alpha T_{o,r} + \beta D_o + \gamma Q_o + \eta E_{o,r} + \lambda C_{o,r} + \mu P_{o,r}. \quad (36)$$

The same edit may therefore be executed differently depending on context. A private face edit may keep detector and local diffusion on the user's device. A batch of product-image recolors may use deterministic projection locally. A high-quality SDXL structural edit may be sent to a cloud GPU if latency and privacy constraints permit.

This model also supports workflow-level batching. If several edits target the same semantic layer, the detector can run once and variants can reuse the result. If multiple images require the same detector backend, the detector model can remain resident. If several low-risk color edits are pending, deterministic projection can run immediately while diffusion jobs are queued. These scheduling opportunities are hidden when each edit is represented as a standalone prompt.

9.3. Quality-aware policy learning

The current rule-based policy is interpretable but incomplete. A learned policy could predict the probability that an operator will satisfy the user request under a time budget. Training data would contain tuples

$$(\ell_i, a, s, \text{model}, \text{crop}, \text{operator}) \rightarrow (T, D, B, H), \quad (37)$$

where H is a human or model-assisted quality score. The policy could then estimate a Pareto frontier rather than a single scalar decision. For example, projection-only may be Pareto-optimal for speed and locality, while local diffusion may be Pareto-optimal for realism, and naive regeneration may be Pareto-optimal for global coherence.

The heavy benchmark provides the systems side of this dataset: timing, drift, boundary, crop, and model features. Reliable semantic and perceptual ground truth is still missing. A later experiment could add human ratings or carefully controlled VLM judgments for edit success, seam quality, identity preservation, and prompt adherence. These labels would support a data-driven workflow scheduler.

9.4. From prototype to reusable workflow artifact

The current repository packages the main reproducibility artifacts: manuscript source, benchmark scripts, reports, aggregate metrics, model-case CSV files, and selected image grids. A more complete artifact would add a formal workflow specification, exact dependency lockfiles or a container, and model hashes where redistribution terms permit.

The image domain supplies a demanding case study, but the underlying contribution is adaptive workflow execution over semantic, cached, provenance-rich intermediate state.

10. Threats to Validity

Semantic correctness. The benchmark uses detector-produced masks as operational targets. It does not yet compare against hand-labeled mask IoU or Dice scores. Therefore, the locality results measure preservation relative to the detected mask, not necessarily semantic correctness relative to human annotation.

Quality metrics. The benchmark reports time, drift, inside change, and boundary change. These are useful systems metrics, but they do not replace human perceptual evaluation, task success judgments, CLIP/BLIP semantic alignment, LPIPS perceptual distance, or VLM-based quality assessment. A diffusion edit can preserve outside pixels and still look visually poor inside the target.

Hardware generality. The run used a local Apple MPS environment. GPU memory behavior, img2img throughput, and detector latency may differ on CUDA, cloud GPUs, edge accelerators, or HPC nodes. The sensitivity projection varies detector and diffusion scaling factors, but it is not a substitute for a measured multi-tier deployment.

Model set. The benchmark includes 10 compatible local checkpoints but skips incompatible files. Newer diffusion architectures and optimized inference engines may change the latency balance.

No watt-level energy measurement or multi-tier deployment. The objective includes resource cost, but the current experiment does not measure power draw or execute across multiple resource tiers. The benchmark instantiates resource cost as a compute-work proxy based on rendered area, step count, and detector invocation. The compute-continuum claims are therefore limited to the workflow representation, measured local operators, and resource-sensitivity scheduling projection.

11. Conclusion

This paper presented semantic layer workflows for adaptive generative AI editing. The core idea is to treat an image-editing session as a provenance-aware workflow over semantic layers rather than as a sequence of independent diffusion calls. The Image DOM stores named masks, boxes, provenance, cache keys, version history, and dependency context. A policy chooses between full regeneration, local diffusion, deterministic projection, cached re-edit, and rollback.

The 200-sample benchmark gives a realistic result. Semantic local rendering is faster than naive regeneration when detector overhead is excluded, but first edits are not universally faster once GroundingDINO+SAM2 is included. The strongest results are workflow-level: cached semantic re-edits average 7.041x speedup, deterministic projection averages 1227x for simple color edits, rollback averages 0.188 s, and semantic compositing preserves non-target pixels under the implemented metric. These results

support the claim that semantic workflow state is valuable for repeated, local, provenance-sensitive generative AI workloads.

Future work should add hand-labeled mask evaluation, CLIP/BLIP and LPIPS scoring, human quality ratings, watt-level energy measurement, and measured multi-tier scheduling experiments. A natural next technical step is to persist and reuse latent denoising trajectories, allowing the policy to restart from selected timesteps and preserve unchanged latent regions, rather than only caching masks and image-space layers.

Data and code availability

The benchmark scripts, generated metrics, reports, manuscript source, and representative image artifacts are available in the public project repository:

github.com/steliosot/semantic-layer-workflows-fgcs

The repository includes the benchmark summary files, model-by-case matrix, selected paper figures, citation metadata, SHA-256 checksums for release assets, and a release archive containing the full metrics JSON and the 200 sample-grid images used to inspect the experimental outputs. Large diffusion checkpoints and external detector weights are not redistributed with the manuscript because they are subject to separate model licenses; the repository records the model identifiers and provides the code required to reproduce the workflow when those dependencies are obtained from their original sources.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

CRediT authorship contribution statement

Stelios Sotiriadis: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review and editing.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During manuscript preparation, the author used ChatGPT/Codex to assist with drafting, LaTeX organization, and summarization of locally produced benchmark results. The author reviewed and edited the output and takes full responsibility for the content.

Appendix A. Representative Image Generations

The appendix includes representative sample grids from the benchmark. Each grid shows the base image, mask/overlay, naive edits, semantic local outputs, deterministic projection or cached variants where applicable, and rollback/version artifacts. The public repository release archive contains the 200 sample-grid images, while Figures A.5–A.10 show representative examples in the manuscript.

Appendix B. Model-by-Case Matrix

The full model-by-case matrix is provided as a CSV file with the manuscript artifacts. Table B.9 reports the detector-inclusive and cached speedups by model and category in compact form.

References

[1] E. Deelman, D. Gannon, M. Shields, I. Taylor, Workflows and e-science: An overview of workflow system features and capabilities, Future Generation Computer Systems 25 (5) (2009) 528–540. doi:10.1016/j.future.2008.06.012.

[2] C. S. Liew, M. Atkinson, M. Galea, T. F. Ang, P. Martin, J. v. Hemert, Scientific workflows: moving across paradigms, ACM Computing Surveys 49 (4) (2017) 1–39. doi:10.1145/3012429.

[3] R. Ferreira da Silva, R. Filgueira, I. Pietri, M. Jiang, R. Sakellariou, E. Deelman, A characterization of workflow management systems for extreme-scale applications, Future Generation Computer Systems 75 (2017) 228–238. doi:10.1016/j.future.2017.02.026.

[4] A. Hertz, R. Mokady, J. Tenenbaum, K. Aberman, Y. Pritch, D. Cohen-Or, Prompt-to-prompt image editing with cross-attention control, International Conference on Learning Representations (2023).
URL https://openreview.net/forum?id=_CDixzkzeyb

[5] L. Zhang, A. Rao, M. Agrawala, Adding conditional control to text-to-image diffusion models, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 3836–3847.

[6] T. Brooks, A. Holynski, A. A. Efros, Instructpix2pix: Learning to follow image editing instructions, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 18392–18402.

[7] E. Deelman, K. Vahi, M. Rynge, G. Juve, R. Mayani, S. Callaghan, P. J. Maechling, K. Wenger, W. Chen, R. Ferreira da Silva, M. Livny, et al., Pegasus, a workflow management system for science automation, Future Generation Computer Systems 46 (2015) 17–35. doi:10.1016/j.future.2014.10.008.

[8] M. Satyanarayanan, The emergence of edge computing, Computer 50 (1) (2017) 30–39. doi:10.1109/MC.2017.9.

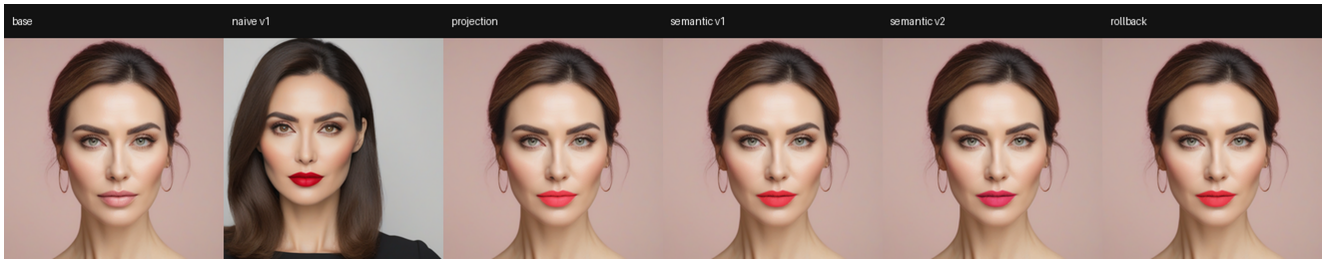

Figure A.5: Representative face/lips edit using Juggernaut XL v2.

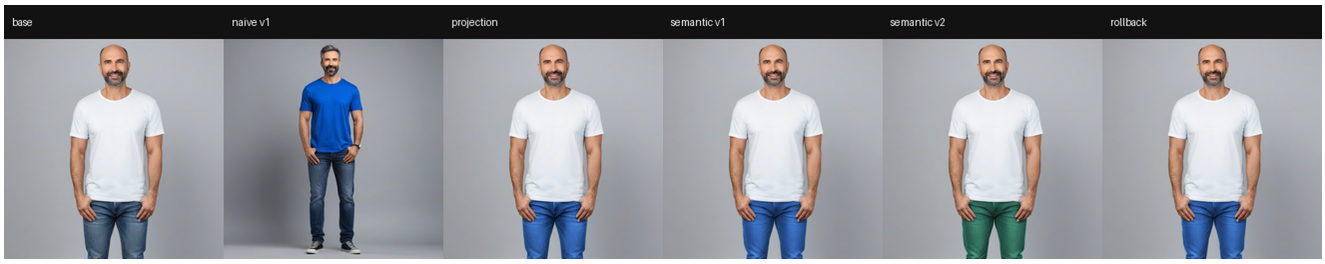

Figure A.6: Representative human body/shirt edit using RealVisXL V4.0.

[9] S. B. Davidson, J. Freire, Provenance and scientific workflows: challenges and opportunities, in: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, 2008, pp. 1345–1350. doi:10.1145/1376616.1376772.

[10] L. Moreau, P. Missier, Prov-dm: The prov data model, W3C Recommendation (2013).
URL https://www.w3.org/TR/prov-dm/

[11] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, et al., The fair guiding principles for scientific data management and stewardship, Scientific Data 3 (2016) 160018. doi:10.1038/sdata.2016.18.

[12] J. Ho, A. Jain, P. Abbeel, Denoising diffusion probabilistic models, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 6840–6851.

[13] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer, High-resolution image synthesis with latent diffusion models, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 10684–10695.

[14] J. Song, C. Meng, S. Ermon, Denoising diffusion implicit models, International Conference on Learning Representations (2021).
URL https://openreview.net/forum?id=St1giarCHLP

[15] C. Meng, Y. He, Y. Song, J. Song, J. Wu, J.-Y. Zhu, S. Ermon, Sdedit: Guided image synthesis and editing with stochastic differential equations, International Conference on Learning Representations (2022).
URL https://openreview.net/forum?id=aBsCjcPu_tE

[16] G. Couairon, J. Verbeek, H. Schwenk, M. Cord, Diffedit: Diffusion-based semantic image editing with mask guidance, International Conference on Learning Representations (2023).
URL https://openreview.net/forum?id=3lge0p5o-M-

[17] R. Tang, L. Liu, A. Pandey, Z. Jiang, G. Yang, K. Kumar, P. Stenetorp, J. Lin, F. Ture, What the daam: Interpreting stable diffusion using cross attention, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, 2023, pp. 5644–5659. doi:10.18653/v1/2023.acl-long.310.

[18] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, L. Zhang, Grounding dino: Marrying dino with grounded pre-training for open-set object detection, arXiv preprint arXiv:2303.05499 (2023).

[19] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollar, R. Girshick, Segment anything, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 4015–4026.

[20] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollar, C. Feichtenhofer, Sam 2: Segment anything in images and videos, arXiv preprint arXiv:2408.00714 (2024).

[21] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, H. Zhao, Depth anything: Unleashing the power of large-scale unlabeled data, arXiv preprint arXiv:2401.10891 (2024).

[22] H. Topcuoglu, S. Hariri, M.-Y. Wu, Performance-effective and low-complexity task scheduling for heterogeneous computing, IEEE Transactions on Parallel and Distributed Systems 13 (3) (2002) 260–274. doi:10.1109/71.993206.

Figure A.7: Representative object/mug edit using DreamShaper 8.

Figure A.8: Representative landscape/sky edit using SDXL Base 1.0.

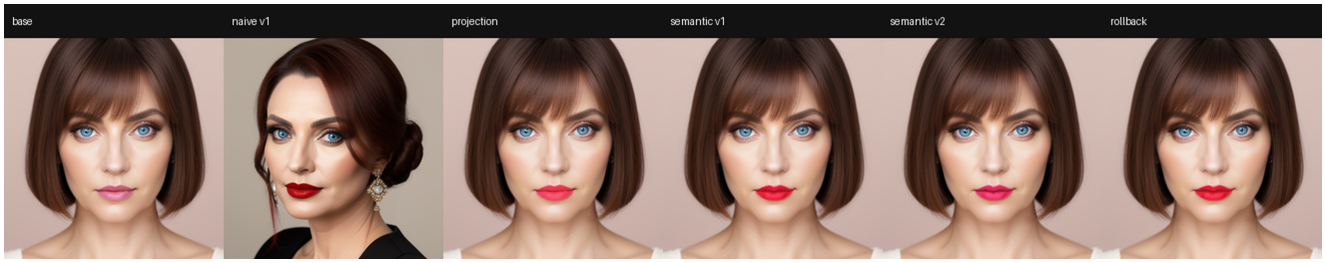

Figure A.9: Representative face/lips edit using Juggernaut Reborn.

Figure A.10: Representative object/mug edit using DreamShaper XL 1.0.

Table B.9: Compact model-by-case speedup matrix. Each cell reports detector-inclusive first-edit speedup / cached re-edit speedup.

Model	Face lips	Human shirt	Object mug	Landscape sky
Deliberate v6	0.598 / 10.814	0.599 / 6.913	0.517 / 4.945	0.604 / 4.303
DreamShaper 8	1.799 / 14.314	1.793 / 5.487	1.734 / 5.793	0.608 / 5.454
DreamShaper XL 1.0	0.630 / 7.138	0.598 / 2.953	0.543 / 4.722	0.511 / 3.982
Juggernaut Reborn	0.662 / 13.084	0.646 / 5.234	0.636 / 5.279	0.628 / 3.893
Juggernaut XL v2	1.165 / 2.917	1.488 / 10.520	1.373 / 15.825	1.184 / 12.794
OpenJourney v4	1.423 / 28.999	1.669 / 11.003	1.385 / 4.827	0.506 / 4.088
RealVisXL V4.0	0.692 / 4.043	0.590 / 3.487	0.711 / 3.149	0.805 / 2.009
Realistic Vision V6.0 B1	0.634 / 11.652	0.475 / 4.511	0.472 / 5.627	0.446 / 3.924
SDXL Base 1.0	0.653 / 6.643	0.554 / 6.090	0.543 / 5.186	0.597 / 4.437
Stable Diffusion 1.5 Base	0.518 / 10.949	0.453 / 4.654	0.463 / 5.228	0.451 / 4.773